

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Technical Presentation

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	35% of CIA	Total Marks:	100
CO1 Statement:	Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	15/07/2026

Code of Conduct

I _____ (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This assessment contains technical presentation. Total: 100 Marks.
- When a student delivers a technical presentation on a data structures topic, evaluation should be based on the following requirements: Concept explanation, Real-world application and Clarity of presentation.
- Demonstrate clear understanding, not just reading slides
- Use of tools: PowerPoint / Google Slides.
- Duration: 7–10 minutes presentation + 3 minutes Q&A.

Section / Topic		Focus	Q Nos	Marks
Data Structure		Real-Time Applications	1	100
GRAND TOTAL:			100 Marks	

Annexure A – Technical Presentation

A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency. (100 Marks)

(OR)

A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation. (100 Marks)

(OR)

A metro rail network maps stations and routes to determine reachable stations from a source station. Suggest a suitable data structure for representation and justify your answer. (100 Mark)

(OR)

A calculator application evaluates arithmetic expressions entered by users. Recommend a suitable data structure and justify based on expression processing efficiency. (100 Marks)

(OR)

A cafeteria token system serves customers strictly in the order of arrival. Identify the suitable ADT and justify why it ensures fairness. (100 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates deep and accurate understanding of data structure with insights and connections	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Content Organization	30	Logical, well-structured, smooth flow; strong introduction and conclusion	Mostly organized with clear flow	Some organization but lacks clarity or transitions	Disorganized, difficult to follow
Technical Depth	20	Includes algorithms, complexity analysis, and advanced insights	Includes algorithm and basic complexity analysis	Limited technical details; missing depth	Minimal or no technical content
PowerPoint Slides	20	Excellent design, clear visuals, enhances understanding	Good slides with minor issues	Basic slides; somewhat cluttered or unclear	Poor slides or no visual support
Communication Skills	10	Clear, confident, engaging delivery; excellent articulation	Clear delivery with minor hesitation	Some hesitation; unclear in parts	Poor communication; difficult to understand

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____



The Calculator's Secret

Efficient Expression Evaluation

Behind every simple calculation lies a clever data structure — and a brilliant algorithm.



The Infix Problem

How Humans Write Math

We naturally use **infix notation**: $A + B * C$ — operators sit between operands.

Why Computers Struggle

Infix requires parsing **operator precedence** and resolving **parentheses** before evaluation order is clear.

The Cost

Complex, multi-pass parsing logic — slow and error-prone without a structured approach.

The Power of Postfix (RPN)



What is Reverse Polish Notation?

In **postfix notation**, operators follow their operands. The expression $A + B * C$ becomes:

$A \ B \ C \ * \ +$

Why It's Superior for Machines

- **No parentheses** — precedence is encoded in position
- **Deterministic order** — left-to-right, single pass
- **Trivial to evaluate** — ideal for stack-based processing

The Engine: The Stack



LIFO Principle

Last In, First Out — the most recently pushed value is the first retrieved. Perfect for nested operations.



Memory Efficient

Requires only **push** and **pop** — basic machine-level instructions with minimal overhead.



$O(n)$ Complexity

Both conversion and evaluation run in **linear time** — each token is processed exactly once.



Transforming the Input

The **Shunting-yard Algorithm** (Dijkstra, 1961) converts infix to postfix in a single pass.



Read Token

**Resolve
Precedence**

Flush Stack

Operands

Sent **directly to output** — no delay, no stack involvement.

Operators

Held on a **temporary stack** until precedence is resolved, then released to output.

Resolution: Computational Efficiency

Evaluating Postfix

01

Push operands

Numbers are placed onto the stack as they are read.

02

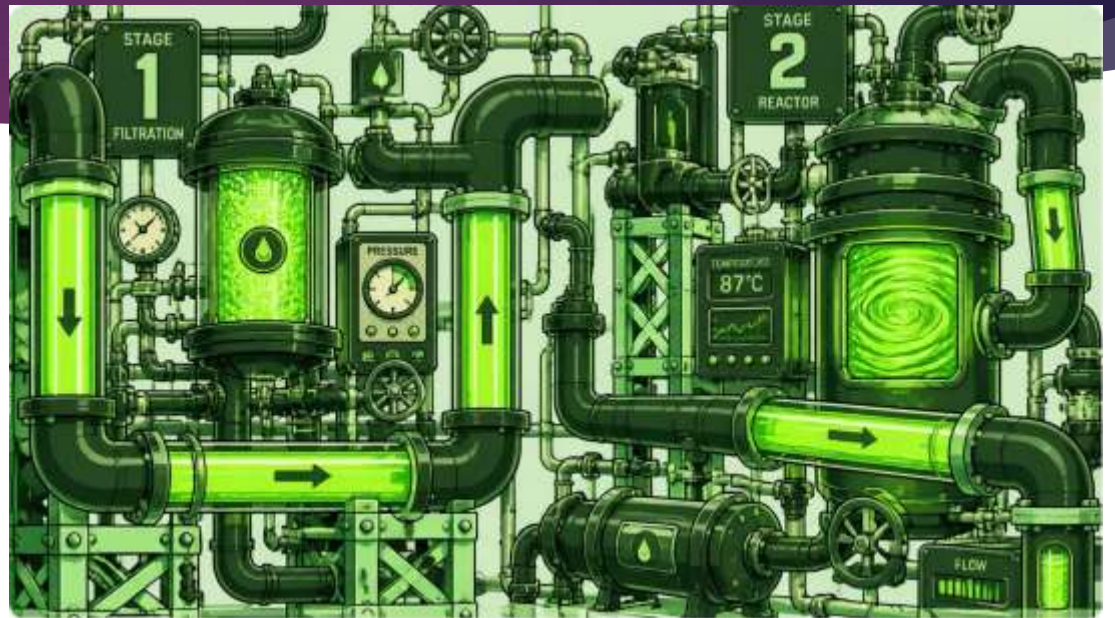
Apply operators

Pop the last two values, compute, push the result back.

03

Final result

A single value remains — the answer.



✓ A two-stage approach — **convert, then evaluate** — delivers maximum efficiency with minimal code complexity. The stack handles everything.

